What to change in FinEx

A practical correction plan for your existing repository, including React web, Flutter mobile and the shared FastAPI backend.

DECISION FROM THE REVIEW

The project has a working React build, a defined API structure and substantial prototype code. It still needs secure identity, durable storage, real-data ingestion and connected client flows before it meets your FinEx implementation plan.

THE MOST URGENT CORRECTION

Production-mode checks accepted an unauthenticated profile request and issued a session for an invalid Google token. Fix identity verification before exposing private user features.

Keep and build on	Replace or complete
Pure black background and orange accents.	Authentication shortcuts and default-user access.
Existing React pages, Flutter screens and API modules.	Hardcoded quotes, random price movements and canned chat replies.
Decimal-based backend calculations and existing tests.	In-memory records, client-owned balances and incomplete mobile flows.

How to use this PDF

- Work through the ordered change packages on page 3. Each package names the files, required behavior and acceptance evidence.
- Use the solo repair schedule on page 17 as a planning target. The full handbook scope remains the project scope.
- Give Antigravity this PDF, the implementation handbook and the approved visual guide. Copy the prompt on pages 18-19.

Repository: github.com/shivrajdhote4452/finex.final

This document specifies changes. It does not claim those changes have already been implemented, deployed or verified.

What was actually verified

Review baseline and limitations. These results describe the checked commit, not future commits or a deployed service.

Check	Observed result
React production build	**Passed**: npm run build completed TypeScript and Vite compilation.
Clean backend installation	**Blocked**: importing the app failed because PyJWT was absent. SQLAlchemy asyncio also required greenlet in this environment.
Backend tests after diagnostic setup	**83 passed, 6 warnings** after installing PyJWT and the SQLAlchemy asyncio extra in the isolated review environment.
Configured lint / formatting gates	Ruff found **163 lint findings**; formatting reported **46 files** needing reformatting.
Production-mode authentication	Anonymous /users/me and /paper-trading/account returned HTTP 200. An invalid Google ID token received an access token.
Paper-order behavior	A LIMIT buy with limit INR 1 filled at INR 2,500. A changed payload using the same idempotency key returned the old order.
Mobile source and build	Flutter source inspected. Android/iOS project folders were absent. Flutter SDK was unavailable, so no mobile build or device test was run.

Interpret the evidence correctly

The 83 passing tests do not establish production readiness: the additional authentication and order checks above exposed behavior the existing suite does not reject. Preserve valid tests and add regression tests for the missing cases.

The local review used Python 3.12. The CI workflow requests Python 3.11. Repeat clean installation and all release gates using the agreed deployment version; the repository currently allows broad dependency version ranges.

LABELS USED THROUGHOUT

Observed: read from the checked code or reproduced locally. **Required change:** the target implementation. **Acceptance:** evidence still to be produced after the fix. No live deployment, cloud setup or store release was verified.

Ordered change packages

CH identifiers belong to this correction plan. They supplement the handbook sprint tasks; they do not replace them.

ID / priority	Required outcome	Detail
CH-01 / Critical	Verified identity; missing or invalid credentials cannot select a user.	p. 4
CH-02 / High	A clean machine installs and starts the declared application dependencies.	p. 5
CH-03 / High	User data survives restarts and is isolated in PostgreSQL/Lakebase.	p. 6
CH-04 / High	Actual permitted market observations with honest freshness metadata.	p. 7
CH-05 / High	Atomic paper execution, correct order rules and durable idempotency.	p. 8
CH-06 / High	React uses the shared API for identity, data and financial operations.	p. 9
CH-07 / High	Flutter provides connected mobile flows and real Android/iOS targets.	pp. 10-11
CH-08 / High	Chat uses retrieved evidence and user-authorized portfolio context.	p. 12
CH-09 / High	Goals, alerts and CSV/manual spending perform durable operations.	p. 13
CH-10 / High	Regression tests, CI, build evidence and truthful release documentation.	pp. 14-16

Dependency sequence

Environment baseline / identity / persistence / real market ingestion / paper execution / React/Flutter integration / complete module behavior / release evidence. Start CH-02 first so CH-01 can be tested immediately.

Build one complete path early: sign in as a real test account, fetch a sourced quote, submit a virtual order, save the ledger, restart the backend and show the same result on web and mobile.

ONE OWNER

Shivraj owns every package. Antigravity should work in small reviewable changes and maintain an open-task list. A screen, success message or passing mock-only test is not sufficient completion evidence.

CH-01: secure authentication

Critical. Remove the path that silently treats unauthenticated requests as Shivraj.

FILES TO INSPECT OR CHANGE

backend/app/routers/auth.py · backend/app/core/auth_stub.py · backend/app/config.py · backend/app/routers/risk.py

OBSERVED

auth.py returns a default user without a Bearer token, accepts mock/test token prefixes and issues an application JWT without verifying the supplied Google token. User profile and consent responses are assembled without durable identity records.

01 **Fail closed.** Missing, malformed, expired or revoked credentials must produce HTTP 401 on private routes. Keep test identity overrides inside tests only; never enable them through a caller-supplied token prefix.

02 **Verify the identity provider.** Validate Google signature, issuer, permitted audience and expiry before accepting claims. Map the verified provider subject to one internal user ID; never trust a body-supplied email as proof of identity. Apply CSRF/state/nonce protections required by the selected sign-in flow. [O1]

03 **Persist the account and session.** Create or load the user using a unique provider/subject pair. Store session and refresh-token state durably; revoke the session on logout and reject refresh-token reuse. Keep server signing secrets out of both clients.

04 **Authorize every user record.** Derive ownership from the verified session. Scope reads and writes for portfolios, watchlists, orders, goals, spending and chat to that owner. Persist questionnaire versions, answers and consent events against that identity.

05 **Wire the UI state.** Start React and Flutter with a session check. Show login when no valid session exists; load onboarding state from the backend. Preserve the requested destination through sign-in.

DONE WHEN

Anonymous and invalid-token calls are rejected in production mode; two signed-in users cannot read or mutate each other’s records; logout survives a restart; profile, consent and onboarding are loaded from stored records.

CH-02: reproducible setup

High. Fix the dependency declarations before interpreting test results or handing the project to another machine.

FILES TO INSPECT OR CHANGE

backend/requirements.txt · backend/pyproject.toml · .github/workflows/ci.yml · web/package.json · mobile/pubspec.yaml

OBSERVED

The code imports jwt, but the declared backend requirements install python-jose instead of PyJWT. In the clean review environment, SQLAlchemy asyncio also lacked greenlet. Installing extra packages manually made tests run, but did not repair the repository manifests.

01 **Choose one JWT implementation.** Either declare PyJWT and retain the current jwt import, or consistently switch all signing/verification code to the chosen existing library. Update tests and manifests together; do not leave an accidental mix.

02 **Declare async database support.** Use the SQLAlchemy asyncio extra so the required greenlet dependency is included on the target platform. Keep asyncpg aligned with the PostgreSQL driver actually used. [O2]

03 **Align supported versions.** Use the same Python version in local instructions, CI and deployment. Record tested dependency versions in the project's dependency strategy; preserve the web lockfile and validate the Flutter lockfile.

04 **Make setup reproducible.** Add a root README with backend, web and Flutter commands, required environment variables, database setup and known blockers. Use .env.example for variable names and safe placeholders, never real credentials.

05 **Retest from an empty environment.** Install using only repository-declared dependencies. App import, database migrations, tests and builds must succeed without undocumented pip installs.

Suggested Windows backend setup after manifest fixes

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r backend/requirements.txt
python -m pip check
python -m pytest -q backend/tests
```

The Python 3.11 command follows the current CI choice. Change it consistently if a different version is selected. These are instructions for the repaired repository, not a claim that the clean baseline already passes.

CH-03: durable storage

High. SQLAlchemy model files alone do not make the active routes persistent.

FILES TO INSPECT OR CHANGE

backend/app/database.py · backend/app/models/ · backend/app/domain/paper_trading_service.py · backend/app/domain/goal_engine.py · backend/app/domain/spending_service.py

OBSERVED

Core business operations write to process-local dictionaries. There are no Alembic migration files in the checked tree. Restarting a process discards those stores; separate workers cannot share them reliably.

Data group	Required durable records / rules
Identity and onboarding	Users, unique provider subject, profiles, consent versions, questionnaire versions, answers, sessions and revocation state.
Markets and portfolios	Canonical instruments, sourced observations, watchlists and items, portfolio ownership, holdings and versioned snapshots.
Paper trading	Accounts, orders, fills, positions, cash ledger and request-key records. Enforce a unique account/key pair and atomic updates.
Goals, spending and alerts	Goals and contributions; uploads, rows and category corrections; rules, events, notifications and delivery attempts.
AI conversations	Owned conversations, messages, evidence references, document versions and feedback.

01 **Create versioned migrations.** Reuse the current models and DDL as inputs, resolve mismatches, then add the Alembic configuration and revision chain. Test upgrade on an empty disposable database and on a previous schema.

02 **Move active routes to repositories.** Inject a database session through the request boundary. Commit related changes together; rollback on failure. Production routes must not fall back to dictionaries when the database is unavailable.

03 **Respect the architecture split.** Keep authoritative transactional records in PostgreSQL/Lakebase. Use Databricks Delta for the approved history and analytics workflows, with replayable exports and lineage.

DONE WHEN

Create data, restart the API, reconnect both clients and retrieve exactly the same owned records. A concurrent-order test against PostgreSQL proves balances and positions cannot be double-spent.

CH-04: actual market data

High. Replace fixture prices and generated history with traceable observations from the approved provider.

FILES TO INSPECT OR CHANGE

backend/app/adapters/market_data_provider.py · backend/app/ports/market_data.py · backend/app/routers/market.py · web/src/context/FinancialContext.tsx

OBSERVED

VERIFIED_OBSERVATIONS contains embedded prices. get_historical_bars derives candles from multipliers. The adapter labels quotes nse_official_report with is_delayed=False. React independently moves prices with Math.random(). These paths do not implement real source ingestion.

01 **Select and document the permitted source.** Use the provider approved in the handbook/source registry. Confirm access, permitted use, coverage, symbols, currency, session calendar and rate limits. If credentials or access are missing, report the blocker and show unavailable data.

02 **Implement a real adapter.** Fetch and validate observations on the backend, map provider symbols to canonical instruments, and store the actual source response or permitted provenance record. Keep raw snapshots and normalized records versioned where required.

03 **Preserve observation time.** Store provider/source, source observation time, retrieval time, currency and freshness status separately. A fresh HTTP response must not make an old market observation appear current.

04 **Use explicit fallback behavior.** Serve an eligible cached observation with its age and stale/delayed state. If no valid observation exists, return unavailable; do not return zero, a fixture or a random price.

05 **Connect history and clients.** Fetch real historical bars and handle gaps, corporate actions and trading sessions under the approved contract. Remove browser/mobile price simulation from production. Both clients render the same backend quote.

DONE WHEN

A displayed price can be traced to a saved provider response and timestamps. Provider failure shows cached/stale/unavailable accurately. Historical bars are sourced records, and a production search cannot resolve to the embedded demo dataset.

Test fixtures remain useful in isolated tests. Keep them visibly separated from production configuration and never attach official-source claims to synthetic values.

CH-05: correct paper execution

High. Virtual trades still require exact, consistent and recoverable accounting.

FILES TO INSPECT OR CHANGE

backend/app/domain/paper_trading_service.py · backend/app/routers/paper_trading.py · backend/app/models/paper_trading.py

Reproduced case	Required behavior
LIMIT buy at INR 1 filled at INR 2,500.	Honor the submitted limit. A non-marketable limit must follow the approved pending/rejection policy, never fill above the buy limit.
Same key, changed symbol/quantity returned the original order.	Compare a canonical request fingerprint. Same key plus different payload must return a conflict without executing.
Replaying an old order after another trade returned a different cash-after value.	Persist and replay the original execution result. Current account balance belongs in the account response, not in a rewritten historical receipt.

Required backend transaction

01 **Validate and bind.** Verify the session and account owner. Validate symbol, side, positive quantity, allowed precision, supported order type and required prices. Reject unsupported types explicitly.

02 **Reserve the request key.** Use a database uniqueness constraint scoped to the account, with a canonical payload fingerprint. Return the saved result for an identical retry; reject conflicts.

03 **Check executable data and funds.** Use an eligible sourced quote under the approved freshness policy. Lock or otherwise safely serialize the account/position updates. Validate available cash and owned quantity.

04 **Commit all effects once.** Calculate with Decimal and the documented rounding/fee policy. Commit order, fill, position, cash movement and audit record in one transaction. A failure must leave no partial trade.

05 **Recover uncertain outcomes.** If the client times out, query the existing request/order status. Reconcile before resubmitting; do not create a new key for the same unresolved user action.

DONE WHEN

Parallel submissions, identical retries, conflicting retries, stale prices, insufficient funds and insufficient holdings all produce the expected ledger outcome. An API restart preserves keys, orders and receipts.

CH-06: connect React to FastAPI

High. Keep the current screens, replace prototype state with server-backed behavior.

FILES TO INSPECT OR CHANGE

web/src/App.tsx · web/src/context/FinancialContext.tsx · web/src/pages/WelcomeScreen.tsx · web/src/pages/OnboardingScreen.tsx · web/src/components/trading/QuickTradeModal.tsx

OBSERVED

App.tsx starts as authenticated. Login uses a timer. FinancialContext seeds balances, holdings and transactions, calculates trades locally and generates price changes. No application API calls were found in the inspected web source.

Area	Required replacement
Session and risk	Fetch session/profile state; perform real Google sign-in and backend session creation; submit questionnaire answers and display the saved deterministic result.
Market and portfolio	Read quotes, history, watchlists and valuations from the API. Display source/freshness. Invalidate the relevant cached data after successful mutations.
Order ticket and ledger	Submit a single intent with a stable idempotency key. Render the backend receipt and ledger. Keep calculation previews visibly separate from authoritative results.
Goals, spending and alerts	Load owned data; persist actions through APIs; preserve typed values on validation/server errors. Re-fetch server totals after writes.
Chat and evidence	Send the actual message to the conversation endpoint. Render only returned answers and traceable evidence; expose insufficient-evidence and provider-error states.

01 **Centralize API access.** Use the frozen OpenAPI contract and generated TypeScript client required by the handbook. Keep base URL, authentication, envelope parsing and error handling consistent.

02 **Keep local state for presentation.** Dialogs, tabs, search text and unsaved forms can remain local. Remove local authoritative balances, fills, risk results and mutation-success simulations.

03 **Verify a complete user journey.** Sign in, inspect a real quote, submit a virtual trade and reload the browser. The account and ledger must match the backend and Flutter.

DONE WHEN

Browser network evidence shows the expected API calls. Refreshing the page preserves committed data. A failed write never displays a success receipt or permanently changes the displayed balance.

CH-07: build the Flutter client

High. Complete mobile integration while preserving the approved black theme.

FILES TO INSPECT OR CHANGE

mobile/lib/main.dart · mobile/lib/theme/terminal_theme.dart · mobile/pubspec.yaml · mobile/lib/screens/ · mobile/test/widget_test.dart

OBSERVED

Flutter opens directly into the main shell and screen state is local. The checked mobile tree contains web and Windows targets, but no android or ios directories. Only a navigation smoke test is present. No Flutter build was run in the review environment.

01 **Add the actual mobile targets.** Generate and review Android/iOS platform configuration in a working branch. Preserve existing Dart screens. Set the application identifiers, environment configuration, callback/deep-link configuration and required platform permissions.

02 **Create the shared API boundary.** Use generated Dart models/client from the same frozen OpenAPI contract as React. Add repositories or services for the feature APIs and consistent envelope/error handling. Reuse existing project conventions where sound.

03 **Implement session lifecycle.** Provide real sign-in, onboarding and consent flows. Store tokens using the platform's secure storage mechanism chosen for the project. Handle expiry, logout, app resume and navigation to a protected destination.

04 **Replace local business operations.** Fetch market data, portfolio, orders, alerts, goals, spending and chat from FastAPI. The backend owns financial calculations and persistent records; mobile stores only appropriate cache and presentation state.

05 **Verify on supported environments.** Run analysis, widget/integration tests and an Android build. Verify iOS on macOS with Xcode and the required signing setup. Record simulator/device/build details and any actual tooling blocker. [O3, O4]

DONE WHEN

An installed Android build uses the real test backend and survives relaunch with the same saved records. iOS build/signing and device-test status are reported accurately. A screenshot or successful Flutter web build does not prove Android/iOS release readiness.

Flutter: screen-by-screen work

Use the mobile section of the visual guide for layouts. This table defines what each screen must actually do.

Flow	Required behavior	Acceptance evidence
Login / consent	Verify identity, save consent version and load onboarding state.	Logout then relaunch: private data stays protected.
Risk questionnaire	Submit complete valid answers and display the saved backend result.	Changing answers changes the result according to approved rules.
Markets / watchlist	Search the actual instrument registry and persist watchlist edits.	Same watchlist and source timestamp appear on web.
Portfolio / overview	Show backend holdings, cash, P&L; and valuation time.	Values agree with the account API after a trade.
Order / review / receipt	Review quantity and amount, submit once, recover unknown status.	Background the app during submission; no duplicate order is created.
Ask FinEx / sources	Send the user question and open returned supporting evidence.	Unrelated questions do not receive the same timed canned answer.
Alerts / notifications	Save rules, show events/history and reflect device permission.	A verified event opens the correct record; permission denial is handled.
Goals / contributions	Create goals and persist contributions with server projections.	Relaunch and web view show the same saved progress.
Spending / CSV	Pick a real file, validate/map, preview rows and confirm import.	Cancel preview: zero transactions added. Confirm: accepted rows only.
Settings / recovery	Persist preferences, end sessions and preserve drafts on errors.	Offline, expired-session, large-text and keyboard flows are usable.

SPECIFIC DEFECT TO REMOVE

spending_screen.dart: _simulateCsvImport() only displays a snackbar claiming rows were parsed. Replace it with actual file selection, backend preview and commit. Success text must reflect the returned import result.

Add missing screens/routes needed by the approved visual guide. Ask Shivraj before designing a new visible layout or changing navigation. Continue API and data work while visual details are awaiting his direction.

CH-08: answers with real evidence

High. Connect the conversation to retrieval, and make every displayed citation auditable.

FILES TO INSPECT OR CHANGE

web/src/pages/AskFinExScreen.tsx · mobile/lib/screens/ask_finex_screen.dart · backend/app/domain/ai_orchestration.py · ai/rag/corpus.py · ai/rag/ingestion.py · ai/rag/retriever.py

OBSERVED

Both clients append predetermined replies after a timer. The backend corpus is embedded text and its ingestion function loops over that data. Keyword retrieval exists, but the inspected paths do not establish extraction from the linked original publications.

01 **Establish source provenance.** Obtain permitted original documents from approved sources. Record original URL, title, publisher, document date, retrieval time, content hash, version and extraction method. Keep the exact passages and their page/section mapping.

02 **Connect the approved retrieval architecture.** Replace embedded example passages with a versioned ingestion pipeline. Implement the retrieval/index components required by the handbook and validate them against a representative question set. Do not relabel keyword search as an embedding model.

03 **Wire the actual question and context.** Send the current conversation and user question through the API. Fetch portfolio metrics from authorized backend records. Keep deterministic financial calculations outside the language model.

04 **Make citations testable.** Return evidence IDs, original text and source locations supporting each answer. When evidence is insufficient or a provider fails, show that state instead of inventing a passage, page or regulatory rule.

05 **Evaluate end to end.** Test answer relevance, citation support, retrieval quality, user-data isolation and prompt-injection resistance. Store model/prompt/index versions and meaningful evaluation results, including failures.

DONE WHEN

Two different questions are processed through the backend, their citations resolve to saved originals, and unsupported questions produce an honest insufficient-evidence result. Remove unverified SEBI/RBI attributions and concentration thresholds from canned UI text.

CH-09: complete the money tools

High. Goals, alerts and spending need actual records and consistent behavior on both clients.

FILES TO INSPECT OR CHANGE

backend/app/domain/goal_engine.py · backend/app/domain/alerts_service.py · backend/app/domain/spending_service.py · backend/app/routers/spending.py · backend/app/routers/goals.py

Goals and contributions

Move goals and contribution history into owned database records. Validate amount, currency and dates; prevent accidental duplicate contributions. Calculate saved progress and projections on the backend, using explicit assumptions. Update both clients after a successful save and preserve input if saving fails.

Alerts and notifications

Persist rules, evaluation events, deduplication/cooldown state and notification history. Evaluate against eligible sourced observations. Use an actual scheduled runner or worker for the approved rule types. Separate in-app notification creation from push delivery status; integrate FCM if required by the handbook and report permission/provider failures accurately.

CSV preview before commit

01 **Parse into a preview.** The current /spending/upload path directly invokes ingestion. Introduce or adapt an approved preview/commit contract: validate headers, encoding, amounts, dates and duplicate rows without adding final transactions.

02 **Show the actual result.** Return valid, duplicate and invalid counts with row-specific errors. Allow column mapping and corrections where required. Replace the mobile fake success message and connect the React import UI.

03 **Commit once after confirmation.** Bind the preview to the authenticated owner, file hash and parsing version. Commit accepted rows atomically; an identical retry must not create duplicate expenses.

04 **Support manual entry and correction.** Provide the missing API action if needed for manual transactions. Persist category changes and provenance. Apply the agreed masking/retention rules before logs, storage and AI access.

DONE WHEN

Canceling a preview writes no transactions; repeated import does not double totals; saved goals, rules and corrected categories survive restart and agree across web/mobile. Do not claim every PII type is masked without representative tests.

Regression tests that must be added

These tests target the observed failures and the integration risks created by replacing local state.

Test	Expected result
Unauthenticated private request	HTTP 401 in production; no default account or profile selected.
Invalid Google token / wrong audience	Login fails; no session or verified user record is created.
Two-user authorization	User B cannot read, change or delete user A's data using guessed IDs.
Session revoke and restart	Revoked session remains revoked after restart and across workers.
Same order key, same payload	One execution and one financial effect; identical saved receipt returned.
Same order key, changed payload	Conflict response; no second execution and no silent old-order substitution.
Non-marketable limit buy	No fill above the limit; approved pending/rejection policy is applied.
Parallel spending of available cash	Concurrent orders cannot overspend; ledger and positions reconcile.
Old receipt after another order	Replay preserves the original cash-after value, timestamp and fill details.
Stale / missing market quote	Accurate data status; no invented history or fallback fixture execution.
API restart and cross-client read	Web and mobile show the same durable order, goal and spending records.
CSV cancel / duplicate commit	Cancel creates zero rows; retries/import duplicates do not inflate totals.
AI source mismatch / no evidence	Unsupported answer is not attributed to a fabricated source or page.
Offline or backgrounded mobile	Drafts survive expected interruptions; uncertain order is reconciled once.

Use fast isolated tests for validation and domain calculations, plus real PostgreSQL integration tests for transactions, ownership and concurrency. Run the important user journeys through the actual API from React and Flutter.

Fixture-backed tests are permitted. A separate integration run must prove the real provider, database and supported device paths; otherwise report those paths as unverified.

CH-10: build and release evidence

Run these gates after the fixes. Preserve full command output with the commit, environment and exit status.

Backend: from the repository root

```
python -m pip install -r backend/requirements.txt
python -m pip check
python -m alembic upgrade head
python -m ruff check backend ai data
python -m ruff format --check backend ai data
python -m pytest -q backend/tests
```

Run migrations against a disposable integration database first. The Alembic command becomes valid after CH-03 adds configuration/revisions. Add the new failure-regression tests before declaring the backend gate complete.

React: from web/

```
npm ci
npm run build
```

Add and document meaningful frontend integration/E2E test commands. The current package has no test script; a successful build alone does not verify login, persistence, order recovery or source display.

Flutter: from mobile/

```
flutter pub get
flutter analyze
flutter test
flutter build apk --release
```

Run the added integration tests on the supported emulator/device. Android release signing and configuration must be prepared; use flutter build appbundle when an app bundle is required. Verify iOS using the documented macOS/Xcode workflow and report its actual result. [O3, O4]

Update CI and documentation

Extend CI beyond the existing backend-only workflow: clean install, lint/format, backend regression and database integration, React build/tests, Flutter analysis/tests and supported build jobs. Fix lint causes instead of globally disabling rules. Update runbooks and UAT claims to match evidence.

EVIDENCE REQUIRED PER CHECKPOINT

Commit/branch; changed files; command and exit code; test counts; environment; screenshots or API results; migration/build artifact identifiers; unresolved blockers. Never convert “not run” into “passed.”

Preserve the design; fix behavior

Your frontend direction remains under your control. This correction work should not become an unrequested redesign.

Keep	Implementation requirement
Pure black app canvas	Keep #000000 for the main background. Preserve the approved neutral dark panels and existing layout unless Shivraj requests a visible change.
Orange primary actions	Keep #F29A60 for primary action/selection. Green and red remain semantic market/status colors, paired with a sign or text.
Approved navigation and screens	Reuse the selected Graphite guide and mobile flows. Ask before inventing a missing visual, changing the sidebar/tabs or adding decorative effects.
Readable financial information	Align amounts and units. Distinguish virtual funds, estimated values, final fills, source observation time and retrieval time.

Replace misleading success and data claims

- Remove “live”, “authentic”, “verified” or official-source badges when the underlying record is a fixture, random simulation or unverified embedded text.
- Do not show a successful Google login, completed import, delivered push notification or filled order until the responsible backend/platform operation has succeeded.
- A loading screen should become data, empty, stale, unavailable or error based on the actual response. Keep a recovery action and preserve valid user input.
- For an unknown order outcome, disable a fresh submission and check the existing order. For expired sessions, re-authenticate and reconcile the pending operation.
- Show source passages and snapshot dates. Do not label static educational text as a personalized, retrieved or regulator-approved recommendation.

ACCEPTANCE

Review desktop and mobile states with real API responses: success, validation failure, empty data, provider outage, expired session and network loss. Verify text scaling, focus, safe areas and keyboard space without changing the chosen visual direction.

Solo repair schedule to 10 November

Start 17 October 2026. These are planning targets for Shivraj, not a promise that unverified work will finish on a date.

Dates / sprint	Main work	Exit evidence
17-20 Oct / R1	CH-02 clean setup; CH-01 verified login, session rules and ownership tests. Reproduce the security defects before replacing them.	Clean dependency install; production anonymous/invalid-token calls rejected; two-user checks.
21-25 Oct / R2	CH-03 migrations/repositories; CH-04 one permitted provider; persist user, quote, portfolio and watchlist records.	Restart proof; provider provenance; no production dictionary fallback; real quote in one client.
26-30 Oct / R3	CH-05 atomic orders and retries; CH-06 React session, data and order integration; durable ledger and recovery.	Concurrency and idempotency tests; sourced-quote-to-order journey survives reload/restart.
31 Oct-4 Nov / R4	CH-07 Flutter targets/API flows; CH-08 evidence-backed chat; CH-09 complete goal, alert and CSV/manual flows.	Cross-client parity; actual CSV preview/commit; mobile test build; traceable chat evidence.
5-10 Nov / R5	CH-10 full regression, remaining integration fixes, device checks, UAT and documentation. Use 8-9 Nov for a candidate freeze and 10 Nov for the handoff.	Reproducible build; release checklist; accurate done/blocked list; demo using real sourced data.

Daily working rhythm

Choose one acceptance outcome each day. Read the current code, add a regression case when behavior is wrong, implement the change, run the focused checks and record evidence. Finish a small coherent commit; update the open-task list before starting the next flow.

IF A GATE SLIPS

Keep the full handbook backlog visible. Do not mark incomplete modules finished or quietly replace real data with mock behavior to meet the date. Report the remaining scope and the exact blocker to Shivraj; agree any scope change explicitly.

External dependencies to resolve early: Google client configuration, database access, permitted market data access, AI/index configuration, push credentials if required, and macOS/Xcode access for iOS verification.

Antigravity master prompt: 1 of 2

Copy this page and page 19 together. Attach the handbook and approved visual PDF as well.

ROLE AND SOURCE OF TRUTH

You are helping Shivraj Dhote repair the existing FinEx repository as its solo developer. Repository: https://github.com/shivrajdhote4452/finex.final.git. Target handoff: 10 November 2026. Treat this correction plan as a supplement to the Combined Implementation Handbook, not a reduced replacement scope.

First inspect the current repository. The review baseline is commit 8c1214e. Compare the current HEAD with that baseline before applying fixes. Identify which findings still exist and which have already been resolved. Read applicable repository instructions. Reuse working components and existing contracts; do not rebuild the project from scratch.

My visual direction is fixed. Keep the approved Graphite Terminal theme with pure black #000000 backgrounds, orange #F29A60 actions and the existing semantic green/red states. Do not use your own design ideas, invent a new layout or redesign approved screens. If a required visible detail is missing, ask me for that detail. Continue independent backend/data work while waiting.

Implement real behavior. React and Flutter must use one FastAPI backend. PostgreSQL/Lakebase is the source of truth for transactional user data. Keep Databricks Delta for the handbook's history and analytics responsibilities. Use generated TypeScript and Dart clients from the agreed OpenAPI contract. Financial calculations and authorization belong on the backend.

No disguised demonstration data. Remove production price fixtures, Math.random() quote changes, timer-based login, canned chat replies and fake CSV-success messages. Use permitted real data sources with traceable timestamps. Keep fixtures only in isolated tests or an explicitly separate demo environment. If a real integration is unavailable, show unavailable and report the blocker.

Work in reviewable increments. Preserve existing work and make focused changes in a working branch. Implement one acceptance outcome at a time. Add regression tests for confirmed defects and keep valid existing tests. Do not weaken authentication, ownership, validation or tests merely to get a green result.

Be precise about completion. Do not claim production readiness, real data, verified citations, successful deployment, store signing or device testing unless the evidence exists. Do not publish a deployment or app-store release unless I have requested that action. Keep credentials in appropriate environment/secret configuration, never in frontend code.

Continue with the ordered implementation instructions on page 19.

Antigravity master prompt: 2 of 2

Continue directly after the previous page. Deliver evidence and an honest remaining-work list.

01 **Repair setup and authentication.** Fix declared JWT and SQLAlchemy asyncio dependencies. Remove default-user and mock-token access from the runtime. Verify Google identity correctly, create durable sessions, persist consent/onboarding and enforce ownership everywhere.

02 **Persist the actual records.** Add and test Alembic migrations; connect the active routes to database repositories. Move balances, orders, ledger, watchlists, goals, spending, alert state and conversations out of in-memory dictionaries.

03 **Connect real market observations.** Implement the approved provider and provenance record. Remove generated historical candles. Distinguish observation time from retrieval time and implement cached/stale/unavailable states accurately.

04 **Correct paper trading.** Enforce supported order types and limit rules. Use transactional account/position updates, durable request-key uniqueness and payload fingerprints. Return the original saved receipt on identical retry; reject a changed payload with the same key.

05 **Wire React and Flutter.** Replace local financial engines and simulated success with real API calls. Add Android/iOS platform configuration. Complete sign-in, risk, portfolio, trade recovery, alerts, goals, spending/imports, settings and evidence reading using the approved visuals.

06 **Complete AI and supporting flows.** Ingest permitted original documents, preserve citation provenance and connect actual user questions to retrieval. Implement two-phase CSV preview/commit, durable contributions, rule evaluation and truthful notification delivery status.

07 **Run the release gates.** Use the regression matrix on page 14. Run clean install, migrations, backend lint/tests, React build and meaningful user-flow tests, Flutter analysis/tests and available platform builds. Verify concurrency and persistence with PostgreSQL.

08 **Report each completed increment.** State the commit, files changed, behavior fixed, commands/exit codes, test results, evidence and remaining blockers. Update the task matrix and runbooks based on actual results, not intended behavior.

START NOW

Begin by inspecting HEAD and reproducing the missing-dependency and authentication failures locally. Implement the smallest complete repair, verify it, then proceed in dependency order. Ask only for genuinely missing credentials, design decisions or unavailable infrastructure; do not repeatedly ask permission for routine reversible implementation work.

Sources and final completion gate

Repository references are pinned to the reviewed commit so later changes do not silently change the evidence.

Reviewed commit: 8c1214eed86c08a143e1e757fd1e9dfce5ffa08c

R1 / Authentication: backend/app/routers/auth.py

R2 / Market adapter: backend/app/adapters/market_data_provider.py

R3 / Order engine: backend/app/domain/paper_trading_service.py

R4 / Client financial state: web/src/context/FinancialContext.tsx

R5 / Web chat: web/src/pages/AskFinExScreen.tsx

R6 / Mobile CSV action: mobile/lib/screens/spending_screen.dart

R7 / Backend dependencies: backend/requirements.txt

R8 / CI workflow: .github/workflows/ci.yml

Supporting project requirements: FinEx Combined Implementation Handbook v1.0 (10 Aug 2026), the solo implementation plan and the approved Graphite Terminal visual guide. Review observations came from local source inspection, build output and targeted local checks; no deployed-service audit was performed.

Official implementation references

O1 / Google token verification - checked 16 Oct 2026

O2 / SQLAlchemy asyncio - checked 16 Oct 2026

O3 / Flutter Android release - checked 16 Oct 2026

O4 / Flutter iOS release - checked 16 Oct 2026

Only mark the project ready when

- Identity and ownership are verified; persisted data survives restart and remains consistent across web and mobile.
- Market records and AI evidence have traceable sources; virtual orders reconcile and recover safely from retries or interruption.
- The agreed handbook flows work end to end, with passing regression/build evidence and accurate Android/iOS verification status.
- Documentation names the delivered commit, setup steps, actual known limitations and every remaining incomplete item.

IMMEDIATE NEXT ACTION

Give Antigravity pages 18-19 and this complete PDF. Start with clean dependencies and production authentication; then verify the first complete real-data user journey.